

Natural Language Processing

Sesión I

Rafael Sánchez, Ph.D.
contacto@rafaelsanchez.dev

Tokens

Word2Vec

Embeddings

1

NLP - Sesión I - Contenido

01 Introducción

02 Representación de las palabras

03 Herramientas

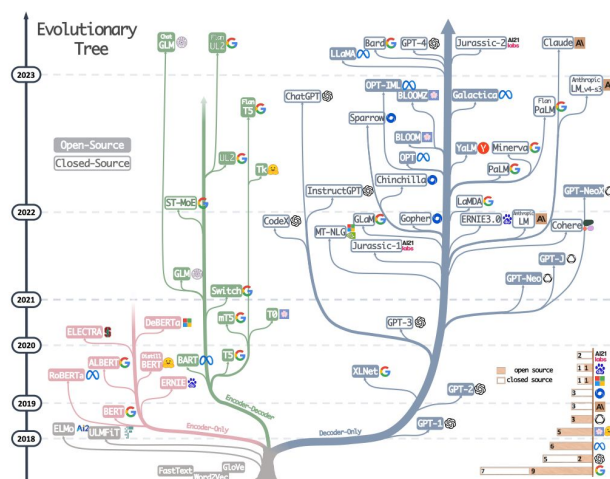
04 Embeddings. Word2Vec

05 Búsqueda de embeddings a escala. ScaNN

¿Qué es el NLP (Natural Language Processing) ?

- El **Procesamiento del Lenguaje Natural** es un campo donde se encuentra la intersección de:
 - Ciencias de la computación
 - Inteligencia Artificial
 - Lingüística
- **Los objetivos** son “entender” el lenguaje para desarrollar tareas útiles para nosotros como por ejemplo:
 - Análisis de textos
 - Traducción automática
 - Respuesta automática a preguntas
 - Resumen de reportes financieros: 10-K, 10-Q, etc
 - Desarrollo de señales de trading: análisis de sentimiento, etc
 - Desarrollo de chatbots
- La **comprensión y representación del significado del lenguaje** o incluso su definición puede ser una tarea muy difícil de solventar.

Evolución de modelos de lenguaje



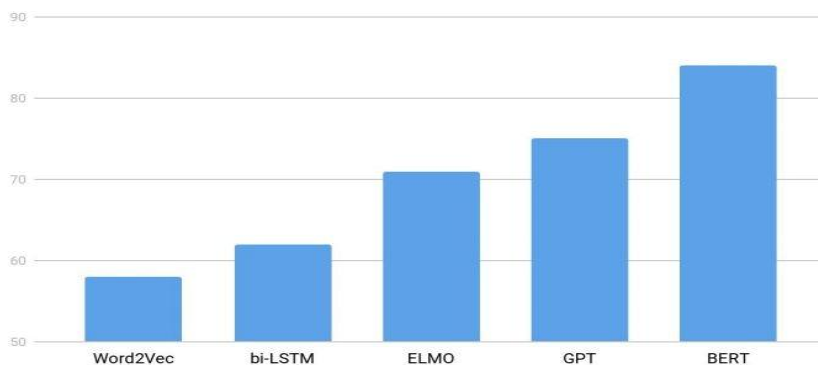
General Language Understanding Evaluation (GLUE)

- GLUE consiste en un grupo de varias tareas NLP, que incluyen clasificación, análisis de sentimiento ó QA entre otras. Cada tarea contiene un dataset de entrenamiento, y un set de evaluación para medir la calidad del modelo. Entre los dataset se incluyen:
 - **CoLA** (Corpus of Linguistic Acceptability): determinar si la sentencia es correcta gramaticalmente.
 - **SST-2** (Stanford Sentiment Treebank): predecir el sentimiento de una frase.
 - **MRPC** (Microsoft Research Paraphrase Corpus): determinar si un par de frases son equivalentes semánticamente.
 - **QQP** (Quora Question Pairs2): determinar si un par de preguntas son equivalentes semánticamente.
 - **MNLI** (Multi-Genre Natural Language Inference): dada una frase y una hipótesis, la tarea es predecir si la frase implica la hipótesis, la contradice, ó es neutral.
 - **QNLI** (Question-answering Natural Language Inference): determinar si el contexto contiene la respuesta a la pregunta.
 - **RTE** (Recognizing Textual Entailment): determinar si la sentencia implica la hipótesis ó no.
 - **WNLI** (Winograd Natural Language Inference): predecir si la frase con el pronombre sustituido está implícito en la frase original.

General Language Understanding Evaluation (GLUE)



Performance on GLUE benchmark (11 tasks) 2018-19



<https://gluebenchmark.com/>

NLP - Sesión I - Contenido

01 Introducción

02 Representación de las palabras

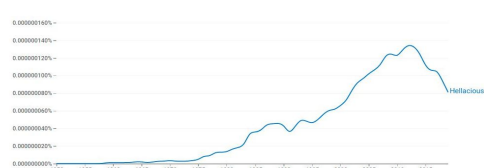
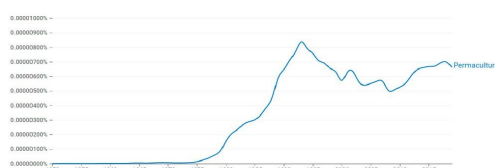
03 Herramientas

04 Embeddings. Word2Vec

05 Búsqueda de embeddings a escala. ScaNN

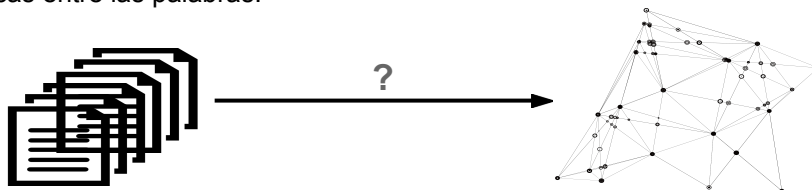
¿Qué tiene de especial el lenguaje humano?

- El lenguaje humano es un sistema construido específicamente para la **comprensión y comunicación** entre los interlocutores ya sea a nivel oral o escrito. No es una señal del entorno, es una comunicación deliberada.
- El lenguaje es un concepto abstracto que varía con el tiempo. Las estructuras y formas de comunicarse no son exactamente iguales con el paso de los años. Un ejemplo de esto es la asimilación o aparición de nuevas palabras por parte del lenguaje.
- El lenguaje humano es principalmente un sistema discreto, simbólico, categórico de señales.
- La codificación de un lenguaje simbólico con grandes vocabularios crea problemas para el machine learning y deep learning: **SPARSITY** (Espacios vectoriales poco densos)



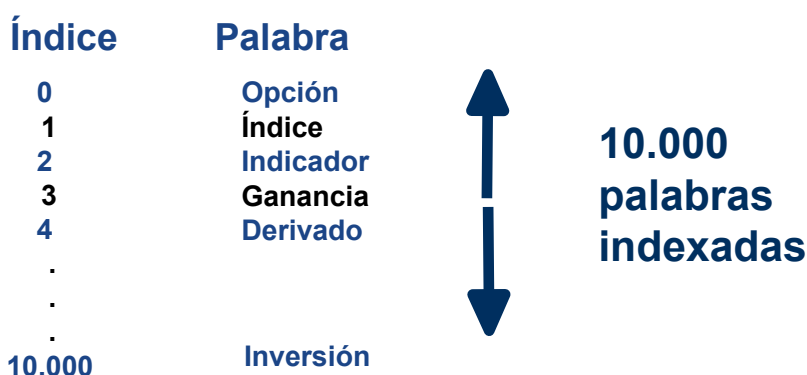
¿Cómo representar las palabras?

- Nuestro objetivo es modelizar el lenguaje para **extraer la información**, **clasificar** los textos, **analizar** el sentimiento, **resumir** los conceptos clave, etc. Sin embargo los modelos **no son capaces de trabajar con conceptos abstractos** como las palabras.
- Nuestro objetivo final es obtener una representación rica que exprese las relaciones semánticas entre las palabras.



¿Cómo representar las palabras? Diccionario Numérico

- Una aproximación naive es a cada una de las palabras de nuestro corpus mapearla con **un único ID**.
Imaginemos que tenemos un corpus con 10.000 palabras.



¿Cómo representar las palabras? One-hot encoding

- Cada una de las palabras de nuestro corpus se mapea con un **único ID**. Este ID está representado por un vector numérico de 0 y 1.

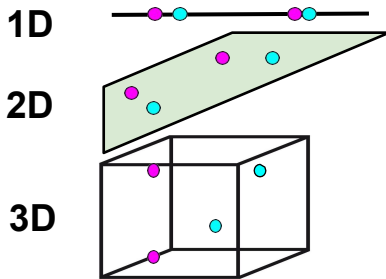
Palabra	Opción	Índice	Indicador	Ganancia	Derivado
Opción	1	0	0	0	0
Índice	0	1	0	0	0
Indicador	0	0	1	0	0
Ganancia	0	0	0	1	0
Derivado	0	0	0	0	1

¿Cómo representar las palabras? One-hot encoding

Palabra	Opción	Índice	Indicador	Ganancia	Derivado
Opción	1	0	0	0	0
Índice	0	1	0	0	0
Indicador	0	0	1	0	0
Ganancia	0	0	0	1	0
Derivado	0	0	0	0	1

- El **espacio de representación** aumenta en una dimensión con cada palabra única del corpus.
 - Un tamaño habitual en los vocabularios va desde **10K** hasta **250K palabras**.
- **No** existe una representación de las **relaciones** entre las palabras.
 - La palabra Opción y la palabra Derivado deberían ser más parecidas que la palabra Opción y la palabra Ganancia.

¿Cómo representar las palabras? One-hot encoding



Distancia(Opción, Derivado) =

$$= \sqrt{(1-0)^2 + \dots + (0-0)^2} = \sqrt{2}$$

Distancia(Opción, Ganancia) =

$$= \sqrt{(0-0)^2 + \dots + (0-1)^2} = \sqrt{2}$$

- El **espacio de representación** aumenta en una dimensión con cada palabra única del corpus.
 - Un tamaño habitual en los vocabularios va desde **10K** hasta **250K palabras**.
- **No** existe una representación de las **relaciones** entre las palabras.
 - La palabra Opción y la palabra Derivado deberían ser más parecidas que la palabra Opción y la palabra Ganancia.

¿Cómo representar las palabras? Bag of Words

- Es un método básico para encontrar tópicos en los textos.
- Necesita crear tokens utilizando tokenización.
- El objetivo es contabilizar cada uno de los tokens, por lo que cada texto quedará representado por la frecuencia de sus palabras.
- Cuando más frecuente es una palabra más importancia toma.

1. The colored flower
2. The bee eats the flower
3. The flower eats the bee
4. The bee eats food

	The	colored	flower	bee	eats	food
1.		1	1	0	0	0
2.		0	1	1	1	0
3.		0	1	1	1	0
4.		0	0	1	1	1

¿Cómo representar las palabras? TF-IDF

- **Term Frequency (tf)** mide la frecuencia de aparición de los tokens en los documentos y es el Bag-of-Words que hemos comentado. Un **tf** grande significa que un token es más importante en el documento:

$$tf(t, d) = (\text{count of term } t \text{ in document } d) / (\text{total number of terms in document } d)$$

- **Inverse Document Frequency (idf)** mide cómo de raro un token es. Refleja la cantidad de documentos que contienen la palabra. Es decir: un token que aparece en muchos documentos es menos importante que un token que aparece en pocos documentos. Tokens como “a”, “de” tendrán un **idf** bajo:

$$idf(t) = \log(\text{total number of documents} / \text{number of documents containing term } t)$$

- **tfidf** asigna **valores altos** a palabras extrañas ya que las considera más informativas:

$$tfidf(t, d) = tf(t, d) * idf(t)$$

Ejemplo

- Suponiendo que el término “hipoteca” aparece **20 veces** en un documento “doc” de 100 palabras, entonces:

$$tf(\text{"hipoteca"}, \text{"doc"}) = 20/100 = 0.2$$

- Suponiendo que tengo un conjunto 10000 documentos, y en 100 de ellos aparece el término “hipoteca”, entonces:

$$idf(\text{"hipoteca"}) = \log(10000/100) = 2$$

- Por tanto, para el documento “doc” donde aparece 20 veces, el cálculo de **tfidf** para el término “hipoteca” es:

$$tfidf(\text{"hipoteca"}, \text{"doc"}) = 0.2 * 2 = 0.4$$

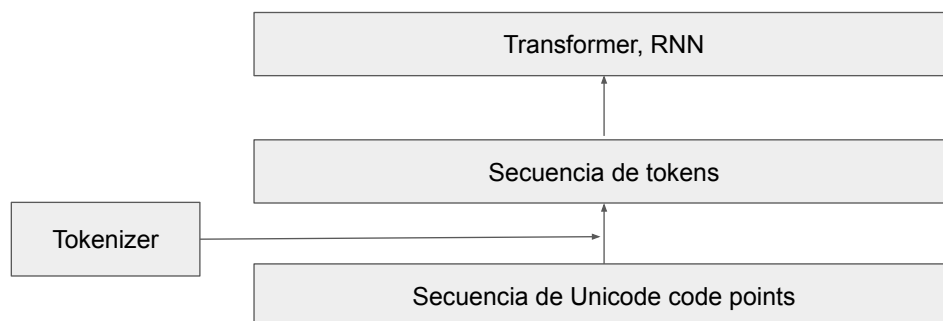
Tokenizers

Un “tokenizer” divide el corpus en elementos más pequeños para ser tratados por los algoritmos de IA. Los “tokenizers” se distinguen en cómo dividen las palabras, manejo de espacios, si consideran mayúsculas o no, etc...

Tokenizers

- ¿ Por qué los LLM son peores en otros idiomas (inglés) ?
- ¿ Por qué los LLM son (eran) malos en aritmética sencilla ?
- ¿ Por qué los LLM no funcionan (ó no funcionaban) bien al generar código ?
- ¿ Por qué los LLM terminan al ver la cadena
"`<|endoftext|>`"?

Tokenizers



Tipos de tokenizers: por carácter y por palabra

- Por carácter:
 - Ventaja: vocabulario pequeño comparado con palabras: 256 caracteres vs. 170K palabras en inglés, pero 20K caracteres en chino. Puede incluir palabras raras fácilmente (OOV).
 - Problema: secuencias muy largas y pérdida de significado por token.
- Por palabra:
 - Ventaja: secuencias cortas
 - Problema: vocabularios muy grandes; muchos OOV

Tipos de tokenizers: por subpalabra

- Por subpalabra:
 - Criterio 1: palabras frecuentes no deberían descomponerse en palabras más pequeñas, aunque podrían descomponerse para extraer significado.
 - Ejemplo: "dog" = "dog"; "dogs" = "dog" + "s"
 - Criterio 2: palabras raras deben descomponerse.
 - Ejemplo: "tokenization" = "token" + "ization"
 - WordPiece: usado en BERT.
 - "##" para indicar token que no es comienzo de palabra
 - Ejemplo: "tokenization" = "token" + "##ization"
 - Unigram: usado en XLNet, Albert.
- Agrupando caracteres en subpalabras:
 - Byte-Pair Encoding: usado en GPT-2, RoBERTa.
 - Tiktoken: usado en GPT-4.
 - SentencePiece: usado en PaLM o Gemini.

Byte-Pair Encoding (BPE)

- Algoritmo inventado por Philip Gage en 1994.
- Algoritmo **iterativo** para reemplazar secuencias de caracteres contiguas más frecuentes por nuevo vocabulario sin usar.
- Ejemplo de codificado (Source: [wikipedia](https://en.wikipedia.org/wiki/Byte_pair_encoding)):
 - Inicial: "aaabdaaabac"
 - Sustitución de "aa" (frecuente) por nuevo vocabulario "z" (= "aa"): "ZabdZabac"
 - Sustitución de "ab" (frecuente) por "Y" (= "ab"): "ZYdZYac"
 - Sustitución de "ZY" por "X" (= "ZY"): "XdXac"
 - Se han creado 3 nuevas palabras de vocabulario. Ya no se puede comprimir más.
 - Para decodificar, se realiza el proceso inverso usando una tabla de "lookup" para las nuevas palabras: "X"="ZY"; "Y"="ab"; "Z"="aa"

SentencePiece and BPE

- SentencePiece es un **tokenizador y detokenizador de texto no supervisado** donde el tamaño del vocabulario se fija antes del entrenamiento:
 - La cantidad de tokens únicos está predeterminada, por ejemplo, 8k, 16k, 32k.
 - Soluciona los problemas de vocabulario infinito en la traducción automática neuronal.
- SentencePiece soporta dos algoritmos:
 - Byte-Pair Encoding (BPE), utilizado pej. en Llama 2 y otros LLM modernos.
 - Modelo de lenguaje Unigram
- **Independiente del idioma:** SentencePiece trata las oraciones como secuencias de caracteres Unicode. No depende del idioma.
- Rápido: velocidad sobre de 50 000 frases segundo y ocupa unos 6 MB.
 - Bueno para entrenamiento e inferencia.

SentencePiece. Training options

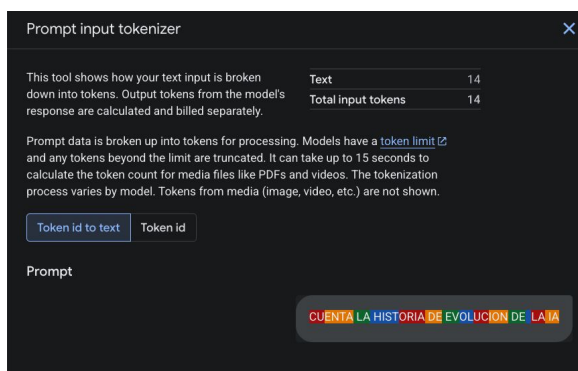
```
options = dict(  
    # input spec  
    input="toy.txt",  
    input_format="text",  
    # output spec  
    model_prefix="tok400", # output filename prefix  
    # algorithm spec  
    # BPE alg  
    model_type="bpe",  
    vocab_size=400,  
    # normalization  
    normalization_rule_name="identity", # ew, turn off normalization  
    remove_extra_whitespace=False,  
    input_sentence_size=200000000, # max number of training sentences  
    max_sentence_length=4192, # max number of bytes per sentence  
    seed_sentencepiece_size=1000000,  
    shuffle_input_sentence=True,  
    # rare word treatment  
    character_coverage=0.99995,  
    byte_fallback=True,  
    # merge rules  
    split_digits=True,  
    split_by_unicode_script=True,  
    split_by_whitespace=True,  
    split_by_number=True,  
    max_sentencepiece_length=16,  
    add_dummy_prefix=True,  
    allow_whitespace_only_pieces=True,  
    # special tokens  
    unk_id=0,  
    bos_id=1,  
    eos_id=2,  
    pad_id=-1,  
    # systems  
    num_threads=os.cpu_count(), # use ~all system resources  
)
```

Tamaño de vocabulario como hiperparámetro

- El vocabulario es un **hiperparámetro** en algoritmos LLM:
 - Configurable para el *tokenizer*.
 - Configurable para las capas de *Embeddings* y en la capa final (*Logits*), pero no hace falta en capas intermedias de los transformers.
- **Vocabulario grande** sería ideal, pero afecta a la necesidad de cómputo (más grande, más cómputo).
- Es posible ampliar el vocabulario de un modelo LLM (pej: tuning), para ello hay que modificar el tokenizador y el modelo.

Demos Tokenizers

- GPT models: <https://platform.openai.com/tokenizer>
- Gemini models: <https://cloud.google.com/vertex-ai/generative-ai/docs/multimodal/list-token>
- Other models: <https://tiktokenizer.vercel.app/>
- Google Cloud AI Studio:



NLP - Sesión I - Contenido

01 Introducción

02 Representación de las palabras

03 Herramientas: Keras para lenguaje natural

04 Embeddings. Word2Vec

05 Búsqueda de embeddings a escala. ScaNN

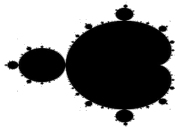
Otros conceptos

- **Estandarización** se refiere al proceso de borrar signos de puntuación ó elementos HTML para simplificar el dataset.
- **Tokenización** se refiere a dividir el texto en elementos más pequeños (*tokens*). Un ejemplo sería dividirlo en palabras individuales usando el espacio en blanco como separador.
- **Vectorización** se refiere a convertir el texto en números para que puedan usarse como entradas a una red neuronal.
- **“Stemming”** se refiere a borrar sufijos de palabras para extraer su base.
- **Lematización** convierte palabras a un diccionario concreto. Requiere más cómputo que “*stemming*”.

Herramientas para trabajar con el texto



TensorFlow
Text



TextBlob



- Eliminación del ruido.
- Normalización del lexicón
- Estandarización
- Tareas específicas: POS tagging, NER tagging, análisis de sentimiento, etc.
- Características estadísticas
- Word Embeddings

tf.keras.utils.get_file

```
tf.keras.utils.get_file(  
    fname=None,  
    origin=None,  
    untar=False,  
    md5_hash=None,  
    file_hash=None,  
    cache_subdir='datasets',  
    hash_algorithm='auto',  
    extract=False,  
    archive_format='auto',  
    cache_dir=None  
)
```

tf.keras.utils.get_file

Downloads a file from a URL if it not already in the cache.

By default the file at the url origin is downloaded to the cache_dir ~/.keras, placed in the cache_subdir datasets, and given the filename fname. The final location of a file example.txt would therefore be ~/.keras/datasets/example.txt.

Files in tar, tar.gz, tar.bz, and zip formats can also be extracted. Passing a hash will verify the file after download. The command line programs shasum and sha256sum can compute the hash.

tf.keras.layers.TextVectorization

```
tf.keras.layers.TextVectorization(  
    max_tokens=None,  
    standardize='lower_and_strip_punctuation',  
    split='whitespace',  
    ngrams=None,  
    output_mode='int',  
    output_sequence_length=None,  
    pad_to_max_tokens=False,  
    vocabulary=None,  
    idf_weights=None,  
    sparse=False,  
    ragged=False,  
    encoding='utf-8',  
    **kwargs  
)
```

tf.keras.layers.TextVectorization

It transforms a batch of strings (one example = one string) into either a list of token indices (one example = 1D tensor of integer token indices) or a dense representation (one example = 1D tensor of float values representing data about the example's tokens). This layer is meant to handle natural language inputs. To handle simple string inputs (categorical strings or pre-tokenized strings) see `tf.keras.layers.StringLookup`.

Note: `tf.keras.preprocessing.text.Tokenizer` does not operate on tensors and is not recommended for new code. Better use `tf.keras.layers.TextVectorization`.

tf.keras.layers.TextVectorization

tf.keras.layers.TextVectorization:

The default `standardization` converts text to lowercase and removes punctuation (`standardize='lower_and_strip_punctuation'`).

The default `tokenizer` splits on whitespace (`split='whitespace'`).

The default `vectorization` mode is `'int'` (`output_mode='int'`). This outputs integer indices (one per token). This mode can be used to build models that take word order into account. You can also use other modes—like `'binary'`—to build bag-of-words models. You will build two models to learn more about standardization, tokenization, and vectorization with `TextVectorization`.

tf.keras.layers.Embedding

```
tf.keras.layers.Embedding(  
    input_dim,  
    output_dim,  
    embeddings_initializer='uniform',  
    embeddings_regularizer=None,  
    activity_regularizer=None,  
    embeddings_constraint=None,  
    mask_zero=False,  
    input_length=None,  
    sparse=False,  
    **kwargs  
)
```

tf.keras.layers.Embedding

Turns positive integers (indexes) into dense vectors of fixed size.

This layer can only be used on positive integer inputs of a fixed range. The `tf.keras.layers.TextVectorization`, `tf.keras.layers.StringLookup`, and `tf.keras.layers.IntegerLookup` preprocessing layers can help prepare inputs for an Embedding layer.

This layer accepts `tf.Tensor`, `tf.RaggedTensor` and `tf.SparseTensor` input.

tf.keras.preprocessing.text.Tokenizer

```
tf.keras.preprocessing.text.Tokenizer(  
    num_words=None,  
    filters='!"#$%&()*+,-./:;<=>?@[\\]^_`{|}~\t\n',  
    lower=True,  
    split=' ',  
    char_level=False,  
    oov_token=None,  
    analyzer=None,  
    **kwargs  
)
```

tf.keras.preprocessing.text.Tokenizer

This class allows to vectorize a text corpus, by turning each text into either a sequence of integers (each integer being the index of a token in a dictionary) or into a vector where the coefficient for each token could be binary, based on word count, based on tf-idf...

By default, all punctuation is removed, turning the texts into space-separated sequences of words (words may include the ' character). These sequences are then split into lists of tokens. They will then be indexed or vectorized.

Self-supervised learning

- El self-supervised learning (SSL) es un tipo especial de aprendizaje de representación que permite aprender una buena representación de datos a partir de un **conjunto de datos sin etiquetar**.
 - Está motivado por la idea de construir tareas de aprendizaje supervisadas a partir de conjuntos de datos no supervisados.
 - El etiquetado de datos es caro y, por lo tanto, el conjunto de datos etiquetado de alta calidad es limitado.
- El aprendizaje de una buena representación facilita la transferencia de información útil a una variedad de tareas posteriores.
 - P.ej. Una tarea posterior con solo algunos ejemplos.
 - P.ej. Transferencia zero-shot a nuevas tareas.
- Las tareas de aprendizaje auto-supervisadas también se conocen como tareas de pretexto.

NLP - Sesión I - Contenido

01 Introducción

02 Representación de las palabras

03 Herramientas

04 Embeddings. Word2Vec

05 Búsqueda de embeddings a escala. ScaNN

Word embeddings

- Una de las principales desventajas de la codificación **one-hot** es que genera **vectores de altas dimensiones y dispersos** que tienen un coste de procesamiento alto.
- Esto se debe a que la codificación *one-hot* genera un vector binario distinto para cada palabra del texto. Además, **no detecta relaciones semánticas** entre palabras, lo que da lugar a modelos deficientes.
- Tampoco TF-IDF detecta relaciones semánticas, ya que se basa en conteo de palabras.

El nacimiento del Embedding neuronal

- La evolución son los **word embeddings**, que genera vectores con dimensiones más pequeñas que sí registran relaciones semánticas entre palabras.
- En 2003, Yoshua Bengio et al. publicaron "*A Neural Probabilistic Language Model*". En el paper se describe cómo **mapear las palabras a un espacio vectorial** utilizando una capa de parámetros entrenables (lookup table) de una red neuronal, iniciando la arquitectura base de los embeddings modernos.

Bengio, Yoshua, Réjean Ducharme, Pascal Vincent, and Christian Janvin. 2003. "**A Neural Probabilistic Language Model**." *Journal of Machine Learning Research* 3 (Feb): 932–38.
<https://www.jmlr.org/papers/v3/bengio03a.html>.

Aplicación de embeddings a múltiples tareas NLP

- Un nuevo paper de Ronan Collobert y Jason Weston aplicó los *word embeddings* a tareas propias de lenguaje natural:
 - Part-Of-Speech Tagging (POS)
 - Chunking
 - Named Entity Recognition (NER)
 - Semantic Role Labeling (SRL)
 - Language Models
 - Semantically Related Words ("Synonyms")
- Fueron los primeros en demostrar el enorme potencial de tomar embeddings pre-entrenados y usarlos de forma unificada para mejorar drásticamente el rendimiento en múltiples tareas de lenguaje natural.

Collobert, Ronan, and Jason Weston. 2008. "A Unified Architecture for Natural Language Processing." In Proceedings of the 25th International Conference on Machine Learning - ICML '08. New York, New York, USA: ACM Press.
<https://doi.org/10.1145/1390156.1390177>.

Word embeddings y Word2Vec

- Un **word embedding** es un **vector multidimensional denso** en el que palabras similares semánticamente tienen una codificación similar. No es necesario especificar el tipo de codificación, ya que un vector de *embeddings* es un vector denso de parámetros entrenables de un modelo (típicamente desde 8 hasta 1024 ó más).
- La idea es generar un vector denso basado en el contexto/uso de las palabras.
- Su construcción puede hacerse de diversas maneras, una de las primeras implementaciones es **Word2Vec**. Es una red neuronal de sólo 2 capas que procesa el texto vectorizando las palabras.
 - **No** es una red neuronal profunda.
 - Procesa el texto de manera que lo puedan entender redes neuronales profundas.
 - Con un conjunto suficientemente grande es capaz de representar las relaciones entre las palabras.

Word Embeddings

- Un **word embedding** es básicamente la representación de las palabras tal como las entienden las personas en un lenguaje que pueda trabajar una máquina. Son una parte esencial para la resolución de los problemas de **NLP**.

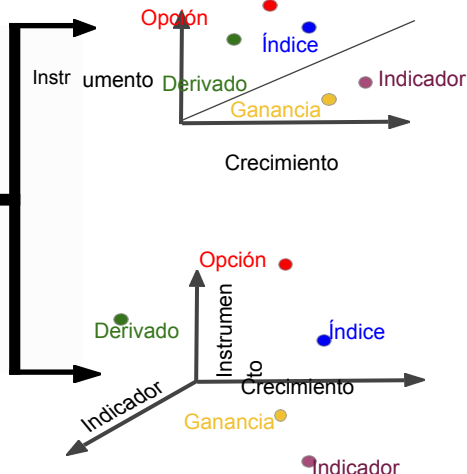
Palabra	D1	D2	D3	D4	D5
Opción	0.95	0.5	0.31	0.3	0.46
Índice	0.93	0.6	0.22	0.4	0.54
Indicador	0.31	0.43	0.1	0.7	0.75
Ganancia	0.21	0.53	0.38	0.12	0.61
Derivado	0.87	0.16	0.37	0.19	0.18



Word Embeddings

- Un **word embedding** es básicamente la representación de las palabras tal como las entienden las personas en un lenguaje que pueda trabajar una máquina. Son una parte esencial para la resolución de los problemas de **NLP**.

Palabra	D1	D2	D3	D4	D5
Opción	0.95	0.5	0.31	0.3	0.46
Índice	0.93	0.6	0.22	0.4	0.54
Indicador	0.31	0.43	0.1	0.7	0.75
Ganancia	0.21	0.53	0.38	0.12	0.61
Derivado	0.87	0.16	0.37	0.19	0.18



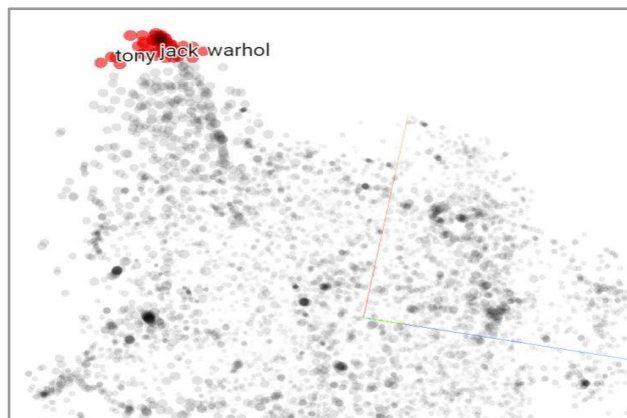
Word Embeddings

- Un *word embedding* es básicamente la representación de las palabras tal como las entienden las personas en un lenguaje que pueda trabajar una máquina. Son una parte esencial para la resolución de los problemas de **NLP**.



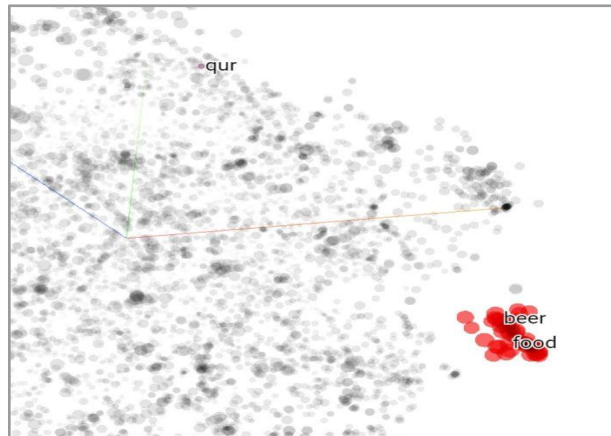
Word Embeddings

- Un *word embedding* es básicamente la representación de las palabras tal como las entienden las personas en un lenguaje que pueda trabajar una máquina. Son una parte esencial para la resolución de los problemas de **NLP**.



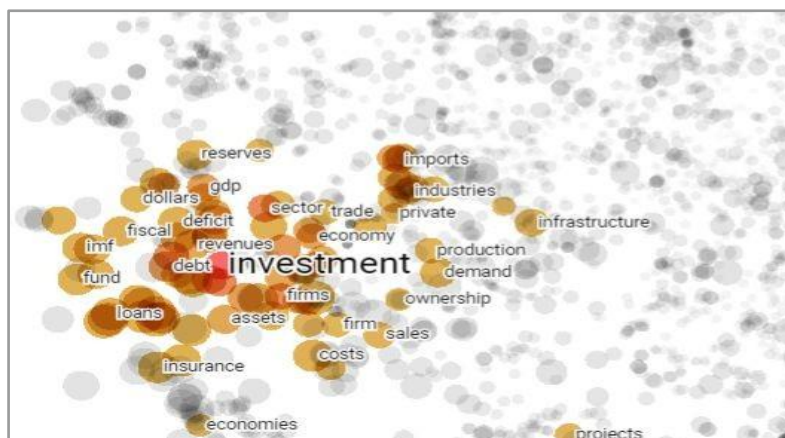
Word Embeddings

- Un **word embedding** es básicamente la representación de las palabras tal como las entienden las personas en un lenguaje que pueda trabajar una máquina. Son una parte esencial para la resolución de los problemas de **NLP**.



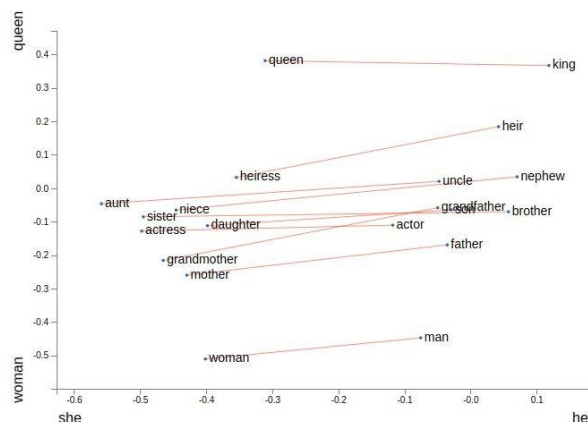
Word Embeddings

- Un **word embedding** es básicamente la representación de las palabras tal como las entienden las personas en un lenguaje que pueda trabajar una máquina. Son una parte esencial para la resolución de los problemas de **NLP**.



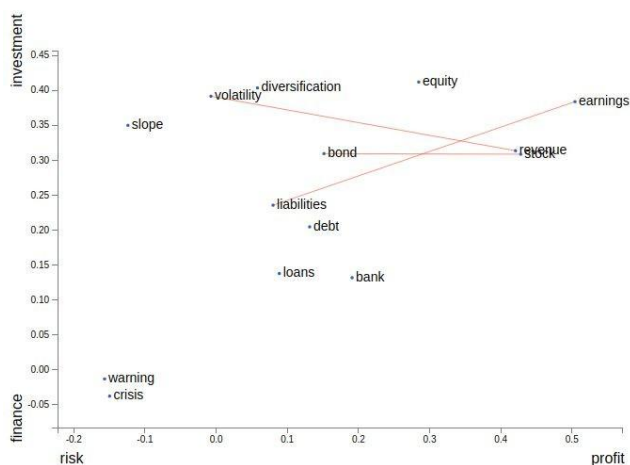
Word Embeddings

- A partir de los Embeddings al obtener representaciones numéricas de las palabras podemos **calcular las similitudes** (palabras-textos) e incluso **realizar cálculos aritméticos** como sumas o restas obteniendo como resultados otras.



Word Embeddings

- A partir de los Embeddings al obtener representaciones numéricas de las palabras podemos **calcular las similitudes** (palabras-textos) e incluso **realizar cálculos aritméticos** como sumas o restas obteniendo como resultados otras.



Terminología de Embeddings

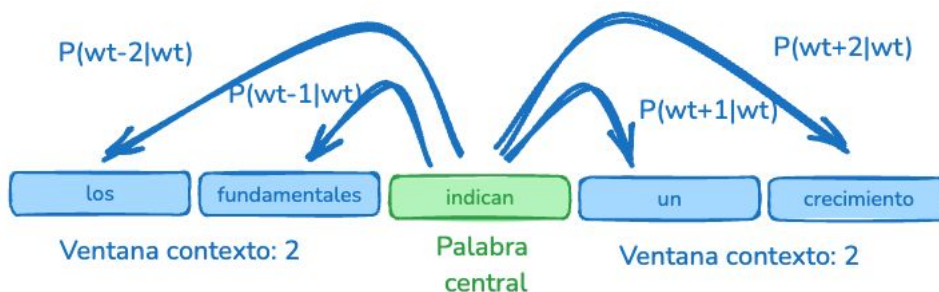
- **Dimensión:** tamaño del vector de embeddings (de 100 a 1000, aunque puede ser incluso mayor):
- **Multi-lenguaje:** capacidad del embedding de mapear el mismo término en distintos lenguajes al mismo espacio latente.
- **Multimodal:** capacidad del embedding de aceptar varios tipos de entrada (vídeo, imagen, ...) y mapearla al mismo espacio latente.
- **Fine-tuning:** capacidad de adaptar el modelo de embeddings una vez entrenado ("*domain specific*").

Word2Vec

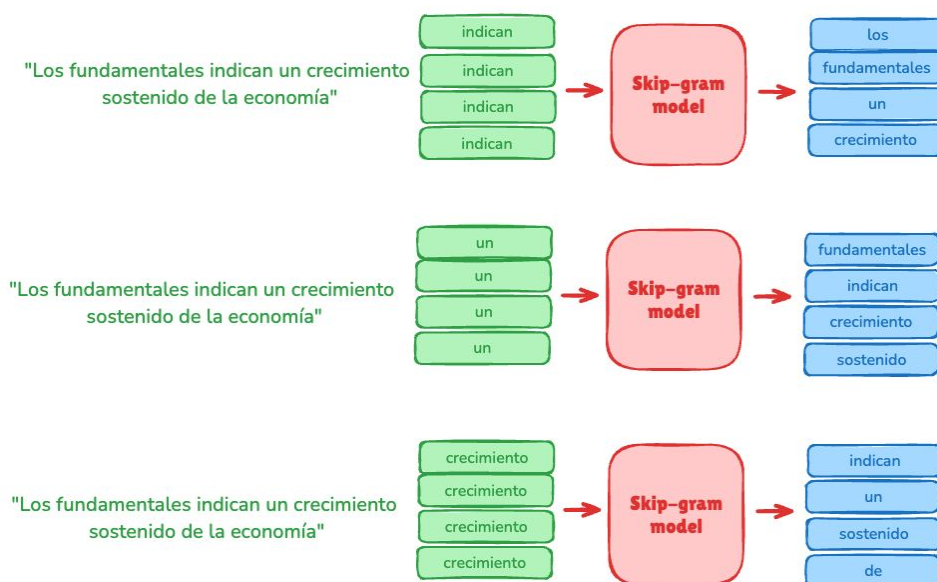
- **Word2Vec** es un algoritmo para el aprendizaje de los vectores palabra o embeddings:
 - Tenemos un gran corpus de texto.
 - Cada palabra en un vocabulario fijo está representada por un vector.
 - Todas las posiciones del texto son una **palabra central** y tienen un **contexto** que las rodean.
 - Se utiliza la similitud de los embeddings de las palabras con su contexto para calcular la probabilidad de la palabra central (o viceversa).
 - Dos variantes: **Skip-Gram** y **CBoW**.
 - Se ajustan los embeddings hasta que se maximiza la probabilidad.

Mikolov, Tomas, Ilya Sutskever, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. "Distributed Representations of Words and Phrases and Their Compositionality," October. <http://arxiv.org/abs/1310.4546>.

Skip-Gram. Ejemplo de funcionamiento



Skip-Gram. Dataset de entrenamiento



Skip-Gram. Dataset de entrenamiento

- Ventana contexto 2 palabras (la ventana real en uso son $2*n+1 = 5$ palabras)
- El dataset de entrenamiento son parejas de palabras (input, target)

Dataset de entrenamiento (parejas de palabras)

(input, target)

indican	los
indican	fundamentales
indican	un
indican	crecimiento
un	fundamentales
un	indican
un	crecimiento
un	sostenido
crecimiento	indican
crecimiento	un
crecimiento	sostenido
crecimiento	de

Skip-Gram. Entrenamiento



Todas las parejas del dataset son 1 epoch.

target	predict	ground	error
el	0.1	0	0.1
los	0.2	0	0.2
fundamentales	0.01	1	0.01
un	0.2	0	0.2
casa	0.3	0	0.3
crecimiento	0.5	0	0.5

Skip-Gram. Entrenamiento

- Para la palabra central “*indican*”, el modelo emite una predicción:
 - Emite tantas predicciones como palabras de vocabulario
- Sabemos el “*ground truth*”, y modificamos los pesos de los vectores
- La epoch concluye cuando hayamos pasado todas las muestras.

Skip-Gram. Entrenamiento. Función Objetivo

- Para cada posición t predecimos las palabras contexto dentro de una ventana de tamaño m dada la palabra central w_j (secuencia de longitud T):

$$\prod_{t=1}^T \prod_{-m \leq j \leq m, j \neq 0} P(w^{(t+j)} | w^{(t)}),$$

- La función objetivo que se minimiza es la siguiente:

$$-\sum_{t=1}^T \sum_{-m \leq j \leq m, j \neq 0} \log P(w^{(t+j)} | w^{(t)}).$$

Minimizar la función objetivo



Maximizar poder predictivo

Mikolov, Tomas, Ilya Sutskever, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. “**Distributed Representations of Words and Phrases and Their Compositionality**,” October. <http://arxiv.org/abs/1310.4546>.

Entrenamiento: Word2Vec Función Objetivo

- Queremos minimizar la función objetivo:

$$-\sum_{t=1}^T \sum_{-m \leq j \leq m, j \neq 0} \log P(w^{(t+j)} | w^{(t)}).$$

- ¿Como calcular $P(w^{(t+j)} | w^{(t)})$?
- Utilizaremos dos vectores por palabra w:
 - v_i cuando w es la palabra central
 - u_i cuando la palabra w es contexto
- Entonces para cada palabra central c tenemos una palabra contexto o:

$$P(w_o | w_c) = \frac{\exp(\mathbf{u}_o^T \mathbf{v}_c)}{\sum_{i \in \mathcal{V}} \exp(\mathbf{u}_i^T \mathbf{v}_c)}$$

Word2Vec

Convierte todo en positivo

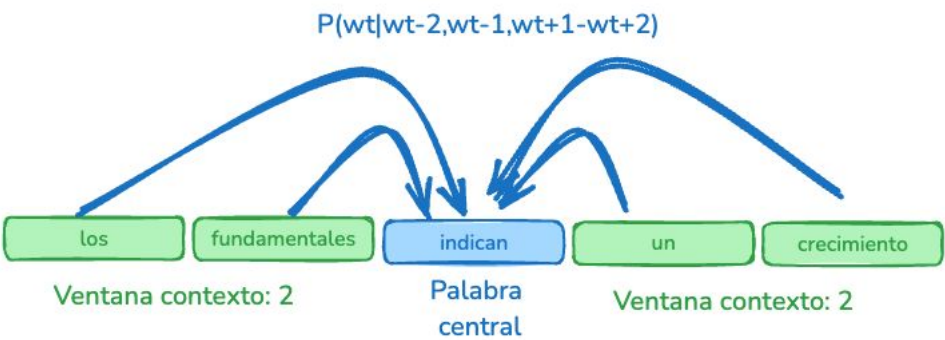
$$P(o|c) = \frac{\exp(u_o^T \cdot v_c)}{\sum_{w \in \mathcal{V}} \exp(u_w^T \cdot v_c)}$$

El producto escalar compara la similitud de o y c. $u^T \cdot v = u \cdot v = \sum_{i=1}^n u_i \cdot v_i$
Grandes productos escalares equivalen a grandes probabilidades.

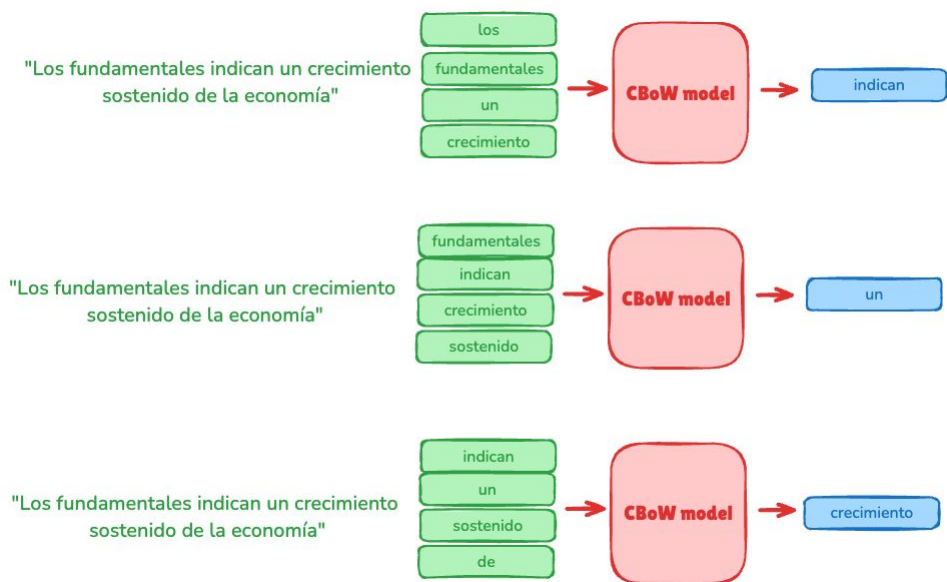
Normaliza todo el vocabulario para dar una distribución de probabilidad

- La función softmax mapea valores arbitrarios x_i a distribuciones de probabilidad p_i :
 - “Max” porque amplifica la probabilidad del valor x_i más alto.
 - “Soft” porque incluso asigna probabilidad a el valor x_i más pequeño.
 - Frecuentemente utilizada en Deep Learning.

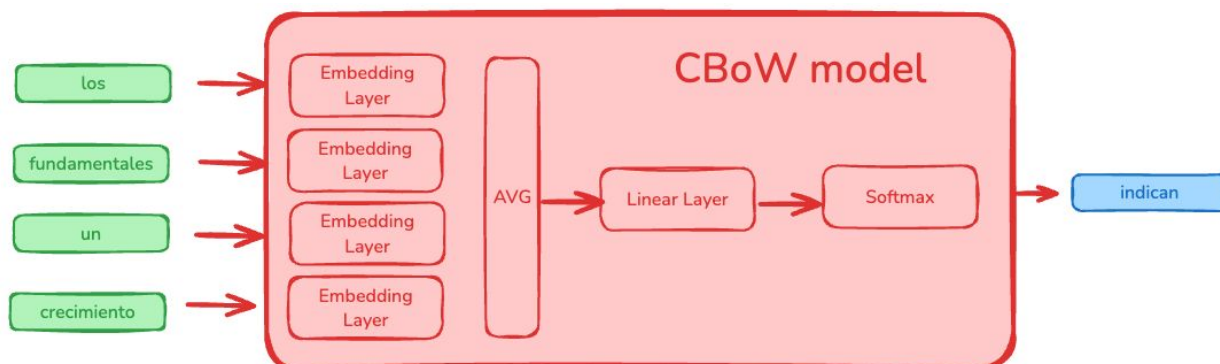
CBoW. Ejemplo de funcionamiento



CBoW. Dataset de entrenamiento



CBoW. Entrenamiento



Negative sampling

- El modelo Word2Vec es un **entrenamiento supervisado**, ya que el dataset de entrada lo forman palabras *context* y *target*. El problema es computacional, ya que el *softmax* hay que implementarlo para **todas las palabras del vocabulario**.
- En vez de dos palabras (*context*, *target*), se puede implementar de otra forma: dado un (*context*, *target*) ¿son una pareja objetivo? (*yes/no*). Es decir, se introduce un tercer elemento en el dataset (*context*, *target*, *yes/no*), de manera que se sustituye muchas funciones *softmax* por muchas funciones *sigmoid*, mucho **más sencillas de implementar**.
- Adicionalmente, se implementa **negative sampling**, implementando varias parejas negativas, típicamente entre 4 y 6, por cada pareja positiva.

Descargar Word2Vec

- Disponible como archivo `.bin` aquí:
<https://drive.google.com/file/d/0B7XkCwpl5KDYNINUTTISS21pQmM/edit>
- Entrenado en 100 billones de palabras del dataset de Google News, con un vocabulario de 3 millones de palabras y frases.
- Vectores de 300 dimensiones
- Archivo `.bin` se puede abrir usando gensim.
- Más información: <https://code.google.com/archive/p/word2vec/>

GLOBAL Vectors (GloVE)

- Método **no supervisado** de generación de embeddings.
- Mientras que Word2Vec es un modelo predictivo, GloVE es un modelo de contar palabras.
- En GloVE, tienes una matriz enorme de $(context, target)$ que ocurren en un corpus de texto.
 - Cada entrada de la matriz corresponde al número de veces que los elementos del vocabulario indicados ocurren juntos.
- GloVE minimiza el error de reconstrucción entre las estadísticas de co-ocurrencia que el modelo estima y las estadísticas de co-ocurrencia reales observadas en el corpus entrenado.
- El problema de GloVE es que ignora palabras que nunca o casi nunca aparecen juntas.

Pennington, Jeffrey, Richard Socher, and Christopher Manning. 2014. "Glove: Global Vectors for Word Representation." In Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing ({EMNLP}), 1532–43. Doha, Qatar: Association for Computational Linguistics. <https://doi.org/10.3115/v1/D14-1162>

Swivel: Submatrix-wise Vector Embedding Learner

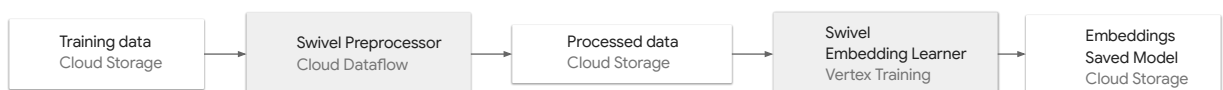
- Encoder **unimodal**
- Genera embeddings de pocas dimensiones a partir de una matriz de co-ocurrencia.
- Resuelve el problema de GloVe de procesar **items raros**: proporciona embeddings de alta calidad para items raros y frecuentes.



Shazeer, Noam, Ryan Doherty, Colin Evans, and Chris Waterson. 2016. "Swivel: Improving Embeddings by Noticing What's Missing." arXiv:1602.02215 [Cs], February. <http://arxiv.org/abs/1602.02215>.

Swivel: Embedding Learning Process

1. **Swivel Preprocessor**: prepara datos al formato requerido para el entrenamiento.
2. **Swivel Embedding Learner**: implementa entrenamiento para aprender embeddings.
3. Los embeddings se sirven típicamente con **un índice ANN** (Approximate Nearest Neighbour)



Shazeer, Noam, Ryan Doherty, Colin Evans, and Chris Waterson. 2016. "Swivel: Improving Embeddings by Noticing What's Missing." arXiv:1602.02215 [Cs], February. <http://arxiv.org/abs/1602.02215>.

¿Por qué Swivel no se menciona tanto hoy en día?

- Swivel fue un hito técnico brillante para optimizar el entrenamiento matricial a gran escala, pero su impacto quedó ensombrecido porque la industria estaba a punto de cambiar de paradigma.
- BERT apareció en 2017, y cambió a **embeddings contextuales ó dinámicos**. La industria abandonó los embeddings estáticos (donde una palabra tiene un sólo vector fijo, ya sea con Word2Vec, GloVe ó Swivel) y saltó a los embeddings contextuales, en donde el vector cambia según la frase, es decir, la palabra “banco” puede tener dos embeddings diferentes.
- Los embeddings dinámicos también dieron paso a los modelos de “Dos Torres” modernos como Sentence-BERT (2019), usados en recomendaciones y búsquedas.

Modelos de Embeddings modernos

- Posteriormente aparecieron los primeros modelos de embeddings modernos basados en transformers están basados en arquitecturas *encoder-only* ó *encoder-decoder*.
- Primeros modelos de embeddings modernos:
 - [NNLM](#): aunque no es un transformer, es una red DNN entrenado en datasets de Google News, optimizado para tareas de lenguaje sencillas como clasificación ó análisis de sentimiento.
 - [Universal Sentence Encoder](#): optimizado para elementos más grandes que palabras, como frases, párrafos ó superiores. La entrada es inglés, y la salida son vectores de 512 dimensiones. Su ventaja es haber sido entrenado para múltiples tareas de NLP que requiere evaluar múltiples palabras (más de una).
 - [BERT](#): optimizado para tareas de NLP que requieren contexto y/o soporte multilinguaje.
 - Modelos como SBERT ó Sentence T5 tenían problemas de generalización con ciertas tareas.
- Un problema de estos modelos es la generación de datasets de calidad para ciertas tareas (Q&A, retrieval, ...)

Modelos de Embeddings modernos

- El uso de LLMs decoder-only ha impactado (positivamente) en la calidad de modelos de embeddings:
 - LLMs se usan ahora para generar datasets de calidad para entrenamiento.
 - Los modelos de embeddings no parten de cero, sino que se inicializan con los pesos de otros modelos.
- Modelos unimodales modernos basados en transformers (decoder-only):
 - text-embedding-005: input: 2048; output: 768
 - text-multilingual-embedding-002: más de 100 lenguajes; input: 2048; output: 768
 - embedding-gemma: input: 2048; output: 128-768; very small, only 308M, multilingual
 - gemini-embedding-001: input: 2048; output: 768-3072, multilingual
 - text-embedding-3-small and text-embedding-3-large: OpenAI models
- Modelos multimodales:
 - gemini-embedding-2-preview: max text 8192 tokens + max 6 images + max 6 PDFMB + 20 sec video, output: 3072
- [MTEB Leaderboard](#): comparativa de modelos de embeddings para distintas tareas

Gemini Embeddings API

- Use an embedding model with **task_type** based on the downstream application for producing better quality embeddings
 - RETRIEVAL_QUERY
 - RETRIEVAL_DOCUMENT
 - SEMANTIC_SIMILARITY
 - CLASSIFICATION
 - CLUSTERING
- Optionally use title for task_type=RETRIEVAL_DOCUMENT

```
from google import genai
from google.genai.types import EmbedContentConfig

client = genai.Client()
response = client.models.embed_content(
    model="gemini-embedding-001",
    contents=[
        "How do I get a driver's license/learner's permit?",
        "How long is my driver's license valid for?",
        "Driver's knowledge test study guide",
    ],
    config=EmbedContentConfig(
        task_type="RETRIEVAL_DOCUMENT", # Optional
        output_dimensionality=3072, # Optional
        title="Driver's License", # Optional
    ),
)
print(response)

# Example response:
# embeddings=[ContentEmbedding(values=[-0.06302902102470398,
# 0.00928034819662571, 0.014716853387653828, -0.028747491538524628, ... ],
# statistics=ContentEmbeddingStatistics(truncated=False, token_count=13.0))]
# metadata=EmbedContentMetadata(billable_character_count=112)
```

Gemini Embedding

- Single embedding model for multiple tasks. Specifying a task instructions can optimize the embeddings.
- A `task_type` parameter specifies the type of task.

task_type	Description
RETRIEVAL_QUERY	Specifies the given text is a query in a search/retrieval setting.
RETRIEVAL_DOCUMENT	Specifies the given text is a document in a search/retrieval setting.
SEMANTIC_SIMILARITY	Specifies the given text will be used for Semantic Textual Similarity (STS).
CLASSIFICATION	Specifies that the embeddings will be used for classification.
CLUSTERING	Specifies that the embeddings will be used for clustering.

Gemini Embedding

Lee, Jinhyuk, Feiyang Chen, Sahil Dua, Daniel Cer, Madhuri Shanbhogue, Iftekhar Naim, Gustavo Hernández Ábrego, et al. 2025. "Gemini Embedding: Generalizable Embeddings from Gemini." arXiv [Cs.CL]. arXiv. <http://arxiv.org/abs/2503.07891>.

Google DeepMind

Gemini Embedding: Generalizable Embeddings from Gemini

Jinhyuk Lee¹, Feiyang Chen², Sahil Dua³, Daniel Cer⁴, Madhuri Shanbhogue⁵, Iftekhar Naim⁶, Gustavo Hernández Ábrego⁷, Zhe Li⁸, Kailu Chen⁹, Hritique Schechter Vera¹⁰, Xiaoqi Ren¹¹, Shaoxing Zhang¹², Daniel Salt¹³, Michael Harsanyi¹⁴, Jay Han¹⁵, Riad Chen¹⁶, Shou Huang¹⁷, Vikram Ravi¹⁸, Paul Suganthan¹⁹, Teng Han²⁰, Andreas Douranoglou²¹, Nithi Gupta²², Feder Moser²³, Cathy Yip²⁴, Ansh Jain²⁵, Simon Baumgartner²⁶, Shikun Shih²⁷, Frank Palus Guter²⁸, Sander Marlowe²⁹, Mike Chen³⁰, Parashar Shah³¹, Sonam Gonska³², Ka Chen³³, Yu Xia³⁴, Scott Chen³⁵, Sai Mohar Karthik Duddu³⁶, Yichang Chen³⁷, Trevor Walker³⁸, Wenli Zhou³⁹, Raksh Ghosh⁴⁰, Zach Gleicher⁴¹, Karen Gill⁴², Zhe Deng⁴³, Mojtaba Seyedhosseini⁴⁴, Yunhsuan Sung⁴⁵, Raphael Hoffmann⁴⁶ and Tom Duerig⁴⁷
Gemini Embedding Team, Google*

In this report, we introduce Gemini Embedding, a state-of-the-art embedding model leveraging the power of Gemini, Google's most capable large language model. Capitalizing on Gemini's inherent multilingual and code understanding capabilities, Gemini Embedding produces highly generalizable embeddings for text spanning numerous languages and textual modalities. The representations generated by Gemini Embedding can be precomputed and applied to a variety of downstream tasks including classification, similarity, clustering, ranking, and retrieval. Evaluated on the Massive Multilingual Text Embedding Benchmark (MMTEB), which includes over one hundred tasks across 250+ languages, Gemini Embedding substantially outperforms prior state-of-the-art models, demonstrating considerable improvements in embedding quality. Achieving state-of-the-art performance across MMTEB's multilingual, English, and code benchmarks, our unified model demonstrates strong capabilities across a broad selection of tasks and surpasses specialized domain-specific models.

1. Introduction

Embedding models, which transform inputs into dense vector representations, are pivotal for capturing semantic information across various domains and modalities. Text embedding models represent words and sentences as vectors, strategically positioning semantically similar texts in close proximity within the embedding space (Gao et al., 2021; Le and Mikolov, 2014; Reimers and Gurevych, 2019). Recent research has focused on developing general-purpose embedding models capable of excelling in diverse downstream tasks, including information retrieval, clustering, and classification (Cer et al., 2018; Murrenigloff et al., 2023). Leveraging their vast pre-training knowledge, large language models (LLMs) have emerged as a promising avenue for constructing such general-purpose embedding models, with the potential to significantly enhance performance across a broad spectrum of applications (Anil et al., 2023a,b; Brown et al., 2020).

The integration of LLMs has revolutionized the development of high-quality embedding models through two primary approaches. Firstly, LLMs have been employed to refine training datasets by generating higher quality examples. Techniques such as hard negative mining (Lee et al., 2024) and synthetic data generation (Jin et al., 2022; Wang et al., 2023) enable the distillation of LLM knowledge into smaller, more efficient embedding models, leading to substantial performance gains. Secondly, recognizing that the embedding model parameters are frequently initialized from language models (Devlin et al., 2019; Karpathis et al., 2020), researchers have explored leveraging LLM parameters directly for initialization (Ni et al., 2021). While this approach introduces increased

*See Contributions and Acknowledgments section. †Equal contributions.

arXiv:2503.07891v1 [cs.CL] 10 Mar 2025

Gemini Embedding architecture

- Based on Gemini model.
- Uses a curated dataset also generated by Gemini:
 - This curated dataset facilitates Gemini embedding model to learn powerful representations.
- Gemini Embedding model was trained with a **noise-contrastive estimation (NCE)** loss with in-batch negatives.
- Excellent multilingual and code understanding capabilities,

Lee, Jinhyuk, Feiyang Chen, Sahil Dua, Daniel Cer, Madhuri Shanbhogue, Iftekhar Naim, Gustavo Hernández Ábrego, et al. 2025. "Gemini Embedding: Generalizable Embeddings from Gemini." arXiv [Cs.CL]. arXiv. <http://arxiv.org/abs/2503.07891>.

Generate Text Embedding Using LLM in BigQuery

1. Run **CREATE MODEL** in BigQuery with **CLOUD_AI_TEXT_EMBEDDING_MODEL_V1** as the **remote_service_type**.

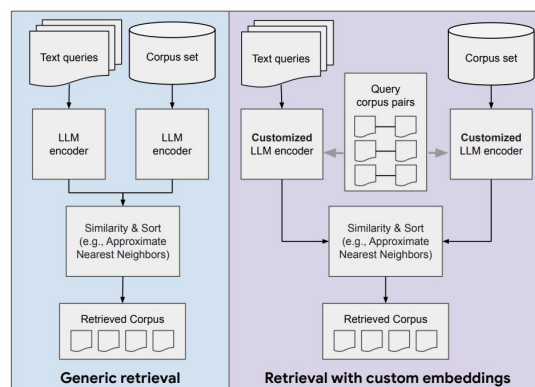
```
CREATE MODEL `bqml_test.cloud_ai_text_embedding_model`  
REMOTE WITH CONNECTION `test_project.us.bqml_test_connection`  
OPTIONS(  
  remote_service_type = 'CLOUD_AI_TEXT_EMBEDDING_MODEL_V1');
```

2. **ML.GENERATE_TEXT_EMBEDDING** to send BigQuery text column to Vertex AI which returns 768-dimension text embeddings.

```
SELECT * FROM ML.GENERATE_TEXT_EMBEDDING(  
  MODEL `bqml_test.cloud_ai_text_embedding_model`,  
  (  
    SELECT content  
    FROM `bqml_test.test_table`  
  ));
```

Tuning an embedding model

- Tune text embedding model to adapt to the embeddings to a **specific domain or task**.
- Example: fine-tune an embeddings model on a specific dataset of customer support tickets for your company. This could help a chatbot understand the different types of customer support issues your customers typically have, and be able to answer their questions more effectively.



Yoon, Jinsung, Serkan O. Arik, Yanfei Chen, and Tomas Pfister. 2023. "Search-Adaptor: Text Embedding Customization for Information Retrieval." arXiv [Cs.LG]. arXiv. <http://arxiv.org/abs/2310.08750>.

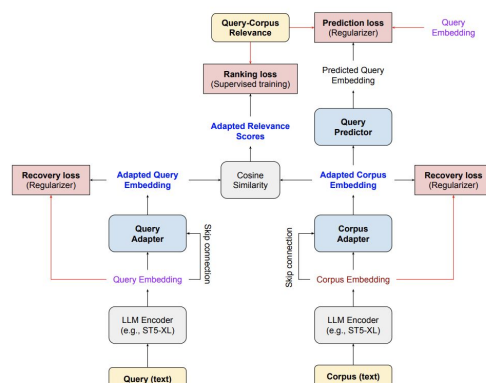
Search-Adaptor: Training

Building blocks of the architecture

- Yellow colored blocks: Given inputs
- Grey colored blocks: Fixed encoders (e.g., pre-trained ST5)
- Blue colored blocks: Additional trainable building blocks
- Red colored blocks: Loss computation

Training

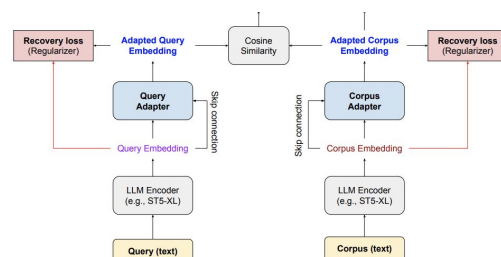
- Utilizing relevance label for supervised training
- Utilizing multiple regularizers
 - Query / Corpus recovery
 - Query / Corpus prediction



Yoon, Jinsung, Serkan O. Arik, Yanfei Chen, and Tomas Pfister. 2023. "Search-Adaptor: Text Embedding Customization for Information Retrieval." arXiv [Cs.LG]. arXiv. <http://arxiv.org/abs/2310.08750>.

Search-Adaptor: Inference

- Search-Adaptor does **not** require full access to the LLM weights.
 - Search-Adaptor modifies the original text embedding generated by pre-trained LLMs, and can be integrated with any LLM, including those only available via APIs.
- At inference time, you only need **query / corpus adaptor** on top of pre-trained LLM encoder
 - Additional parameters is less than 1M params.
- Outputs are the adapted relevance scores between query and corpus.
 - Then, use the scores to sort the relevance corpus per each query.



Yoon, Jinsung, Sercan O. Arik, Yanfei Chen, and Tomas Pfister. 2023. "Search-Adaptor: Text Embedding Customization for Information Retrieval." arXiv [Cs.LG]. arXiv. <http://arxiv.org/abs/2310.08750>.

Multimodal embedding models

- gemini-embedding-2 is a SOTA model for multimodal embeddings.
- Maps **text, image, video, audio and PDFs** into a single, unified vector space, and captures semantic intent across over 100 languages.
- Example: you can embed a web page containing a mix of text, charts and video (the model eliminates the need for intermediate processing steps or separate embedding models).
 - This simplifies complex pipelines and RAG systems.
- Max input length: 8192 tokens (text), 6 images, 120 seconds (video), 6 pages (PDF).
- Dimensions: 3072 (variable)

Multimodal embedding models

```
result = client.models.embed_content(
    model='gemini-embedding-2',
    contents=[
        "The jetpack is cool",
        types.Part.from_bytes(
            data=image_bytes,
            mime_type='image/png',
        ),
    ],
    config=types.EmbedContentConfig(output_dimensionality=3072)) # Optional
)

# This produces two embeddings
for embedding in result.embeddings:
    print(embedding.values)
```

Demo “FindMeMedia” gemini-embedding-2

- FindMeMedia is a sample application showcasing the multimodal capabilities of gemini-embedding-2 model.
- Contains over 1 million images, 500 thousand audio recordings, and 500 thousand video clips, all indexed and searchable across modalities.
- <https://findmemedia.lmm.ai/>

EmbeddingGemma

Vera, Henrique Schechter, Sahil Dua, Biao Zhang, Daniel Salz, Ryan Mullins, Sindhu Raghuram Panyam, Sara Smoot, et al. 2025.

"EmbeddingGemma: Powerful and Lightweight Text Representations." arXiv [Cs.CL]. arXiv.

<http://arxiv.org/abs/2509.20354>.

Google DeepMind

Gemini Embedding: Generalizable Embeddings from Gemini

Jihyuk Lee¹, Feiyang Chen¹, Sahil Dua¹, Daniel Cer¹, Madhuri Shanbhogue², Ihekhar Naim, Gustavo Hernández Abrego, Zhe Li, Kaifeng Chen, Henrique Schechter Vera, Xiaoqi Ren, Shangling Zhang, Daniel Salz, Michael Bonetk, Jay Han, Blat Chen, Shuo Huang, Vikram Rao, Paul Sgaurhan, Jing Han, Andreas Doumanoglou, Nithi Gupta, Fedor Moiseev, Cathy Yip, Ansh Jain, Simon Baumgartner, Shahrokh Shahi, Frank Palma Gomez, Sandeep Marisetla, Min Choi, Parashar Shah, Sonam Goenka, Re Chen, Ye Xia, Koert Chen, Sat Mohar Karthik Duddu, Yuhang Chen, Trevor Walker, Wenzel Zhou, Raksh Gkya, Zach Gleicher, Karan Gill, Zhe Dong, Mojtaba Seyedsheini, Yunhuan Sung, Raphael Hoffmann and Tom Durig
Gemini Embedding Team, Google¹

In this report, we introduce Gemini Embedding, a state-of-the-art embedding model leveraging the power of Gemini, Google's most capable large language model. Capitalizing on Gemini's inherent multilingual and code understanding capabilities, Gemini Embedding produces highly generalizable embeddings for text spanning numerous languages and textual modalities. The representations generated by Gemini Embedding can be precomputed and applied to a variety of downstream tasks including classification, similarity, clustering, ranking, and retrieval. Evaluated on the Massive Multilingual Text Embedding Benchmark (MMTEB), which includes over one hundred tasks across 250+ languages, Gemini Embedding substantially outperforms prior state-of-the-art models, demonstrating considerable improvements in embedding quality. Achieving state-of-the-art performance across MMTEB's multilingual, English, and code benchmarks, our unified model demonstrates strong capabilities across a broad selection of tasks and surpasses specialized domain-specific models.

1. Introduction

Embedding models, which transform inputs into dense vector representations, are pivotal for capturing semantic information across various domains and modalities. Text embedding models represent words and sentences as vectors, strategically positioning semantically similar texts in close proximity within the embedding space (Gao et al., 2021; Le and Mikolov, 2014; Reimers and Gurevych, 2019). Recent research has focused on developing general-purpose embedding models capable of excelling in diverse downstream tasks, including information retrieval, clustering, and classification (Coe et al., 2018; Muennighoff et al., 2023). Leveraging their vast pre-training knowledge, large language models (LLMs) have emerged as a promising avenue for constructing such general-purpose embedding models, with the potential to significantly enhance performance across a broad spectrum of applications (Anil et al., 2023a,b; Brown et al., 2020).

The integration of LLMs has revolutionized the development of high-quality embedding models through two primary approaches. Firstly, LLMs have been employed to refine training datasets by generating higher quality examples. Techniques such as hard negative mining (Lee et al., 2024) and synthetic data generation (Dai et al., 2022; Wang et al., 2023) enable the distillation of LLM knowledge into smaller, more efficient embedding models, leading to substantial performance gains. Secondly, recognizing that the embedding model parameters are frequently initialized from language models (Devlin et al., 2019; Karpukhin et al., 2020), researchers have explored leveraging LLM parameters directly for initialization (Ni et al., 2021). While this approach introduces increased

¹See Contributions and Acknowledgments section. ²Equal contributions.

arXiv:2503.07891v1 [cs.CL] 10 Mar 2025

EmbeddingGemma

- Based on **Gemma 3 architecture**.
- Multilingual support with **100 languages**.
- **Customizable output dimensions**: output dimensions from 768 to 128 for speed and storage tradeoffs using Matryoshka Representation Learning (MRL).
- 2K token window.
- Storage: run it on less than **200MB of RAM** with quantization
- Low latency: **less than 22ms inference** on EdgeTPU for fast and fluid applications.
- Offline and secure: generate embeddings of documents directly on your hardware, works without internet connection.
- Integration with sentence-transformers, llama.cpp, MLX, Ollama, LiteRT, transformers.js, LMStudio, Weaviate, Cloudflare, LlamaIndex, LangChain and more.

Vera, Henrique Schechter, Sahil Dua, Biao Zhang, Daniel Salz, Ryan Mullins, Sindhu Raghuram Panyam, Sara Smoot, et al. 2025.

"EmbeddingGemma: Powerful and Lightweight Text Representations." arXiv [Cs.CL]. arXiv. <http://arxiv.org/abs/2509.20354>.

EmbeddingGemma architecture

- Gemma 3 adapted into an encoder-decoder model following the T5Gemma recipe.
- Initialize EmbeddingGemma from the encoder of this encoder-decoder model, thus getting all the knowledge.
- Hyperparameters:
 - 24 layers
 - Embedding dimension (internal): 768
 - FFN layer: 3072
 - Embedding dimension (output): 768
- During pre-training, each sample includes a query, a positive passage and an optional negative passage.
- Pre-training uses three different losses: noise-contrastive loss, Global-orthogonal regularize-based loss, and embedding matching loss.

Vera, Henrique Schechter, Sahil Dua, Biao Zhang, Daniel Salz, Ryan Mullins, Sindhu Raghuram Panyam, Sara Smoot, et al. 2025. "EmbeddingGemma: Powerful and Lightweight Text Representations." arXiv [Cs.CL]. arXiv. <http://arxiv.org/abs/2509.20354>.

Mobile RAG: EmbeddingGemma + Gemma 3n

```
import torch
from sentence_transformers import SentenceTransformer

device = "cuda" if torch.cuda.is_available() else "cpu"

model_id = "google/embeddinggemma-300M"
model = SentenceTransformer(model_id).to(device=device)

# Query embedding
query_embedding = model.encode(
    "How do I use prompts with this model?",
    prompt_name="Retrieval-query"
)
```

NLP - Sesión I - Contenido

01 Introducción

02 Representación de las palabras

03 Herramientas

04 Embeddings. Word2Vec

05 Búsqueda de embeddings a escala. ScaNN

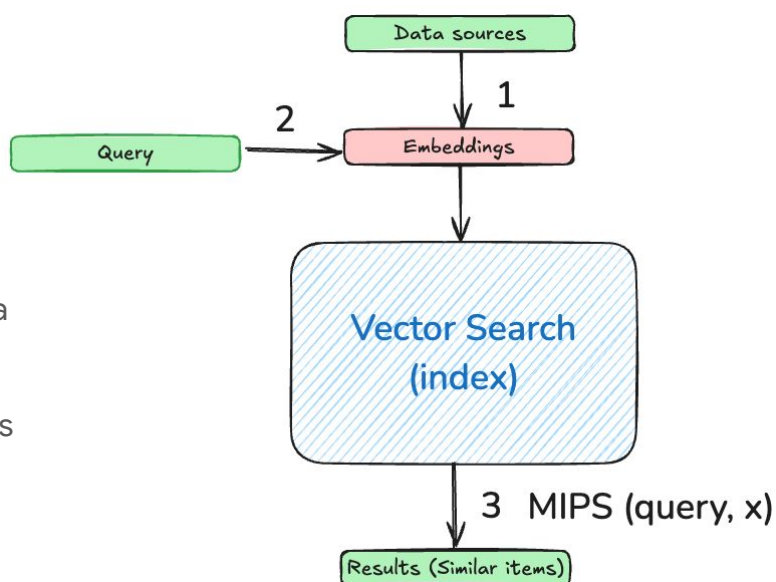
Problema:

**¿ Cómo encontramos
embeddings similares en el
espacio multidimensional de
manera rápida y escalable ?**

Casos típicos de búsqueda vectorial a escala

- Motores de búsqueda
- Motores de recomendación
- Personalización (ads, recomendaciones, ...)
- LLMs y RAG (extraer información de documentos)
- Seguridad: control de spam, ...
- Clasificación/búsqueda de imágenes
- Chatbots / GenAI

Búsqueda vectorial



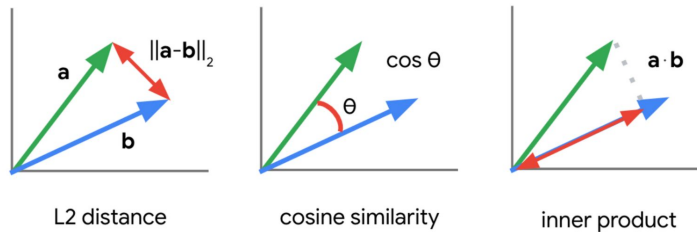
Pasos:

1. Crear índice a partir de un corpus de texto (lleva tiempo)
2. Hacer una query
3. Conseguir los resultados similares

Búsqueda vectorial (Vector Search) no escala bien

Fast and scalable vector search
isn't easy

Brute force search takes too long: $O(\text{dims} \times \text{items})$

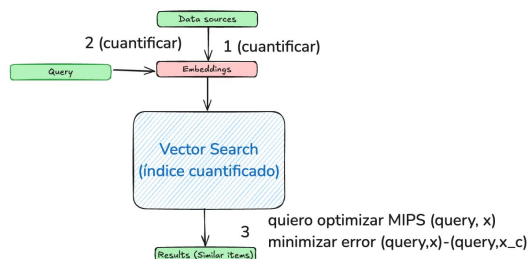


Source:

<https://cloud.google.com/blog/products/ai-machine-learning/how-to-use-grounding-for-your-llms-with-text-embeddings>

Índice ANN (Approximate Nearest Neighbor)

- **Maximum Inner Product Search (MIPS)** es una métrica para encontrar los documentos más relevantes a una query de usuario
 - Para el caso de búsqueda ("*similarity search*"): queries de usuarios y documentos están en el mismo espacio vectorial denso.
 - El cómputo de MIPS es **caro**.
 - Para aliviarlo, se usan técnicas basadas en **cuantificación**.

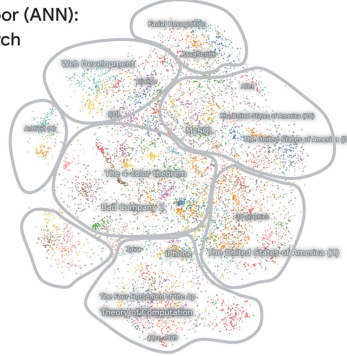


Cuantificación

- Esquemas de cuantificación tradicionales tienen el objetivo de **minimizar la distancia media** entre cada vector x y su cuantificación $\tilde{x}$ (el “centroide”).
- Aunque esta métrica es muchas veces útil, en el caso de búsqueda vectorial **no es equivalente a optimizar el resultado de la búsqueda (MIPS)**
- Cuantificación con distancia mayor (error mayor) puede dar lugar a una precisión superior.

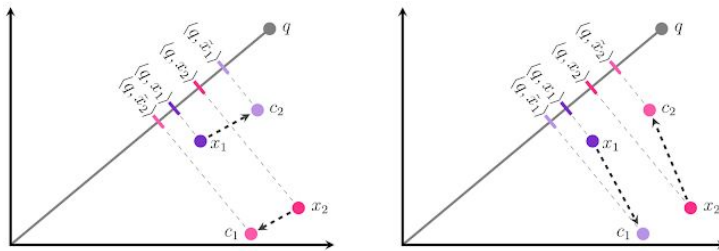
Approximate Nearest Neighbor (ANN):
Fast and scalable vector search

Building an index with Vector Quantization



Cuantificación

- Por tanto, el problema de la cuantificación es que minimizar el error (distancia media) es subóptimo para MIPS. Solución: usar **cuantificación dependiente del valor** (*score-aware quantization*)



- Se trata de optimizar el producto de la query y los resultados
- MIPS mejor si centroides alejados
- Se penaliza la dirección paralela al vector query

Guo, Ruiqi, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. 2020. "Accelerating Large-Scale Inference with Anisotropic Vector Quantization." *arXiv:1908.10396 [Cs, Stat]*, December. <http://arxiv.org/abs/1908.10396>.

Nueva función de pérdida

- **Cuantificación anisotrópica:** nueva función de pérdida donde el error de cuantificación paralelo tiene más peso que el ortogonal.

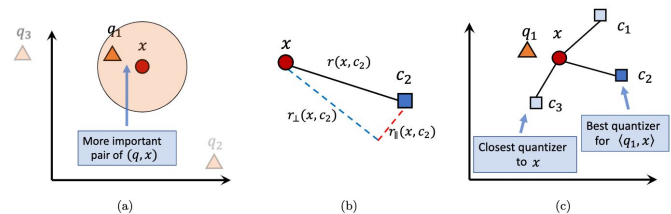


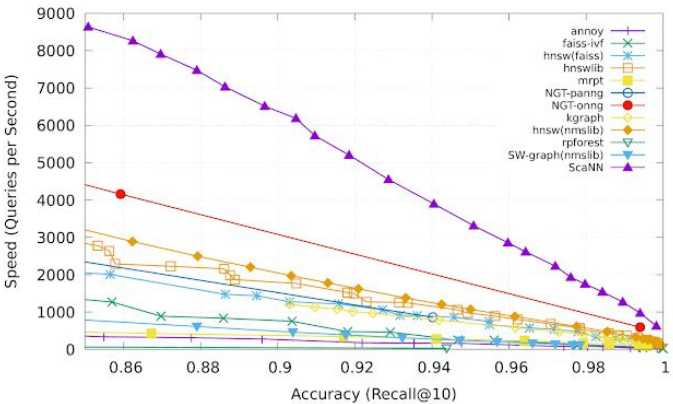
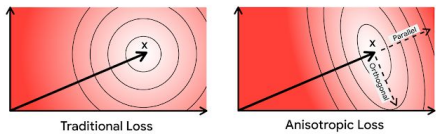
Figure 1: (a) Not all pairs of q and x are equally important: for x , it is more important to accurately quantize the inner product of $\langle q_1, x \rangle$ than $\langle q_2, x \rangle$ or $\langle q_3, x \rangle$, because $\langle q_1, x \rangle$ has a higher inner product and thus is more likely to be the maximum; (b) Quantization error of x given one of its quantizer c_2 can be decomposed to a parallel component $r_{||}$ and an orthogonal component $r_{\perp}$. (c) Graphical illustration of the intuition behind Equation (2). Even if c_3 is closer to x in terms of Euclidean distance, c_2 is a better quantizer than c_3 in terms of inner product approximation error of $\langle q_1, x - c \rangle$. Notice that c_3 incur more parallel loss ($r_{||}$), while c_2 incur more orthogonal loss ($r_{\perp}$).

Guo, Ruiqi, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. 2020. “Accelerating Large-Scale Inference with Anisotropic Vector Quantization.” *arXiv:1908.10396 [Cs, Stat]*, December. <http://arxiv.org/abs/1908.10396>.



Cuantificación anisotrópica de vectores

Mayor precisión con menor latencia



glove-100-angular.com
ann-benchmarks.com

Guo, Ruiqi, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. 2020. “Accelerating Large-Scale Inference with Anisotropic Vector Quantization.” *arXiv:1908.10396 [Cs, Stat]*, December. <http://arxiv.org/abs/1908.10396>.



OSS ScaNN with Keras

```
!pip install -q scann # ScaNN does not have its own separate layer in KerasRS because the ScANN library is TensorFlow-only
from scann import scann_ops

def build_scann(
    candidates,
    k=10,
    distance_measure="dot_product",
    dimensions_per_block=2,
    num_reordering_candidates=500,
    num_leaves=100,
    num_leaves_to_search=30,
    training_iterations=12,
):
    builder = scann_ops.builder(
        db=candidates,
        num_neighbors=k,
        distance_measure=distance_measure,
    )

    builder = builder.tree(
        num_leaves=num_leaves,
        num_leaves_to_search=num_leaves_to_search,
        training_iterations=training_iterations,
    )
    builder = builder.score_ah(dimensions_per_block=dimensions_per_block)

    if num_reordering_candidates is not None:
        builder = builder.reorder(num_reordering_candidates)

    # Set a unique name to prevent unintentional sharing between
    # ScaNN instances.
    searcher = builder.build(shared_name=str(uuid.uuid4()))
    return searcher
```

OSS ScaNN with Keras

```
def run_scann(searcher):
    pred_movie_ids = searcher.search_batched_parallel(
        user_embedding,
        final_num_neighbors=10,
    ).indices
    return pred_movie_ids

searcher = build_scann(candidates=candidate_embeddings)

pred_movie_ids = run_scann(searcher)
```

Fin Sesión I

Rafael Sánchez, Ph.D.
contacto@rafaelsanchez.dev